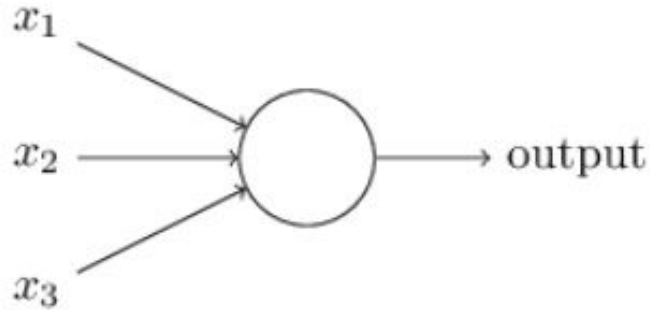
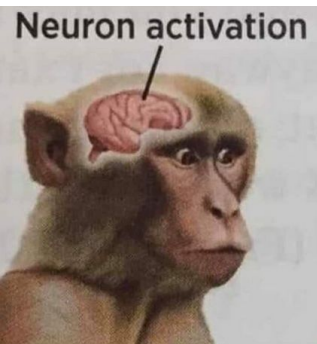

A Hitchhiker's Guide to Neural Networks

Classic Perceptron

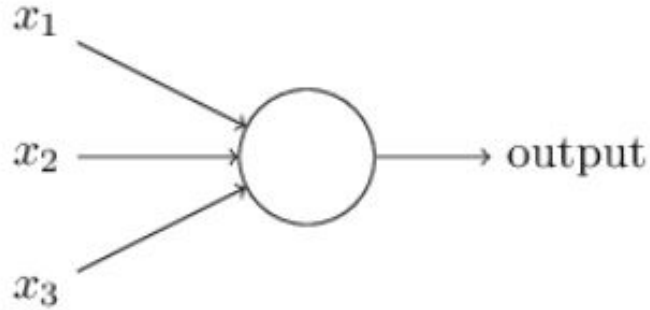


$$\text{output} = \begin{cases} 0 & \text{if } \sum_j w_j x_j \leq \text{threshold} \\ 1 & \text{if } \sum_j w_j x_j > \text{threshold} \end{cases}$$

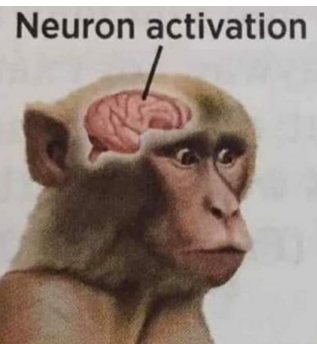


- An atomic way to *make decisions*
- Inspired by actual neurons (yes the things in your brain)

Classic Perceptron - alternate

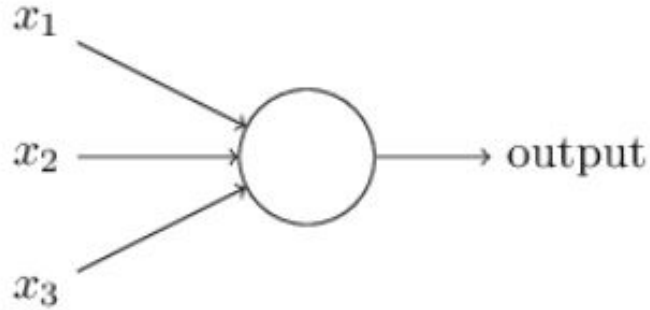


$$\text{output} = \begin{cases} 0 & \text{if } w \cdot x + b \leq 0 \\ 1 & \text{if } w \cdot x + b > 0 \end{cases}$$

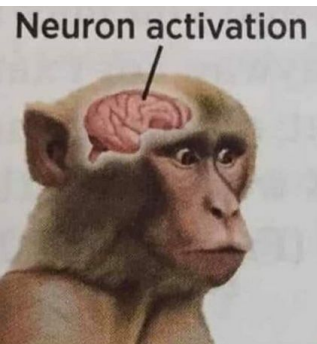


- An atomic way to *make decisions*
- Inspired by actual neurons (yes the things in your brain)

Classic Perceptron - the issue

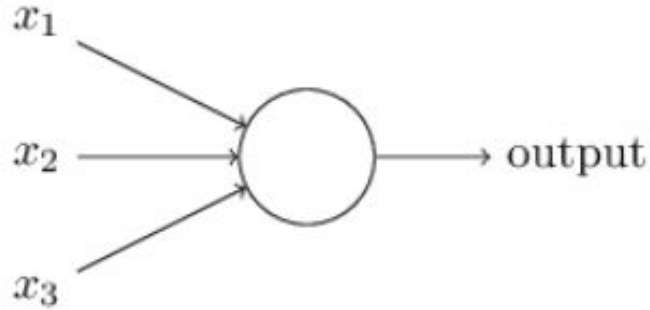


$$\text{output} = \begin{cases} 0 & \text{if } w \cdot x + b \leq 0 \\ 1 & \text{if } w \cdot x + b > 0 \end{cases}$$



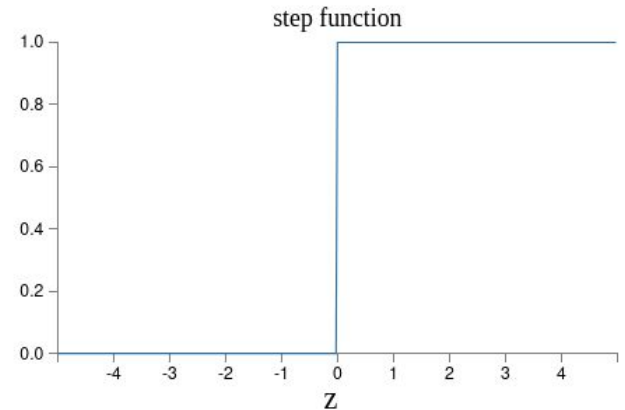
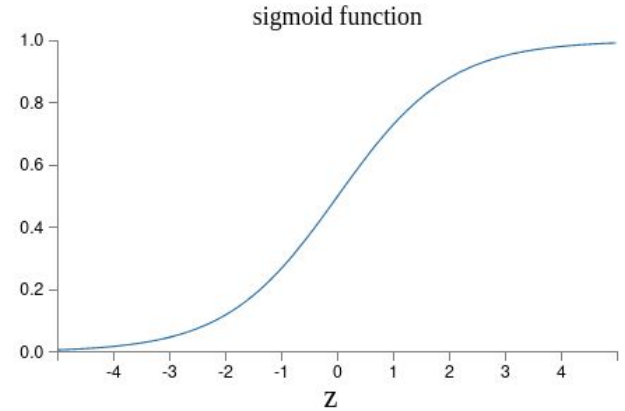
- Problem with this neuron: not differentiable
- Cannot run gradient descent

Sigmoid Perceptron

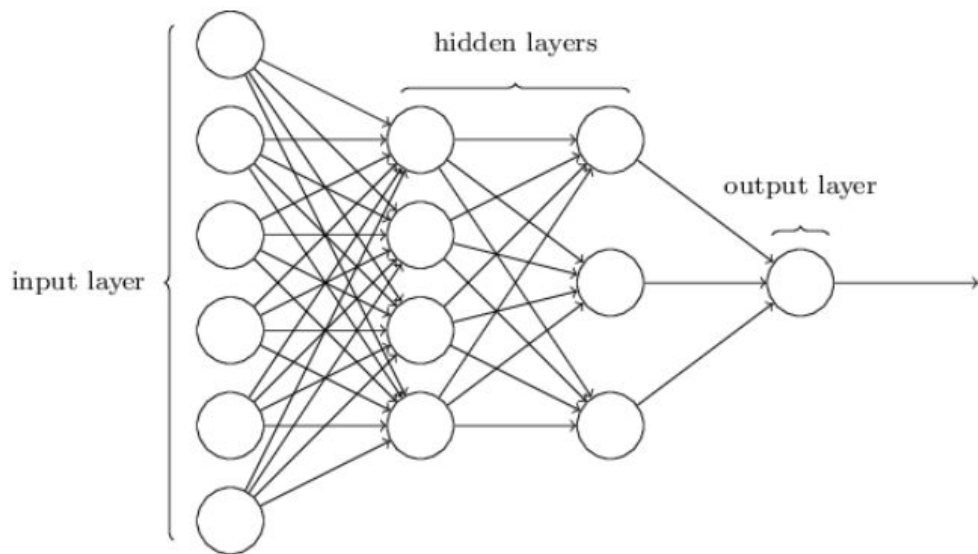


$$\sigma(z) \equiv \frac{1}{1 + e^{-z}} \cdot \rightarrow \text{activation function}$$

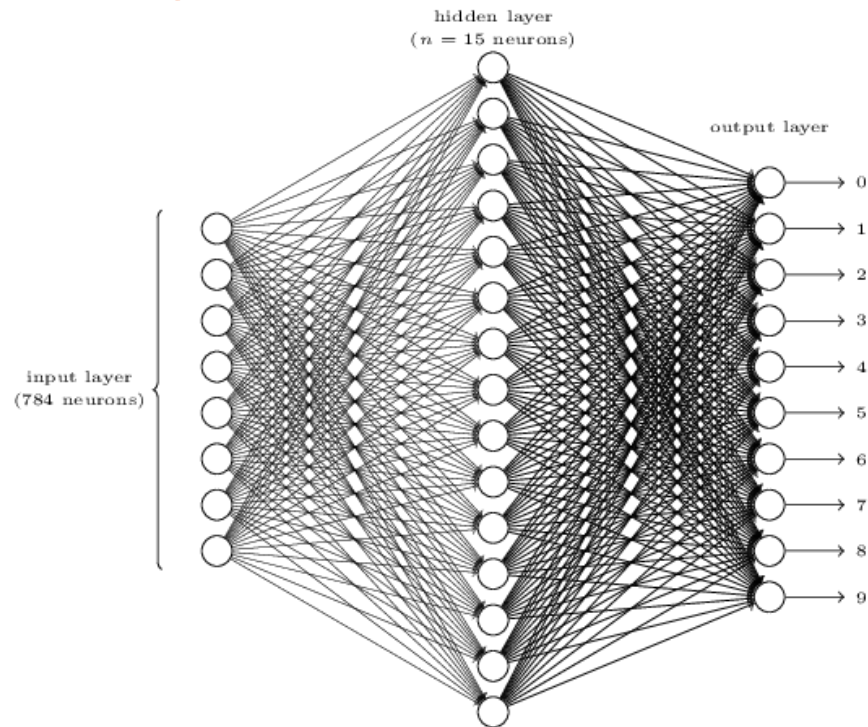
$$\text{output} = \frac{1}{1 + \exp(-\sum_j w_j x_j - b)}.$$



Neural Networks (finally!)



aka multilayer perceptrons (MLPs)

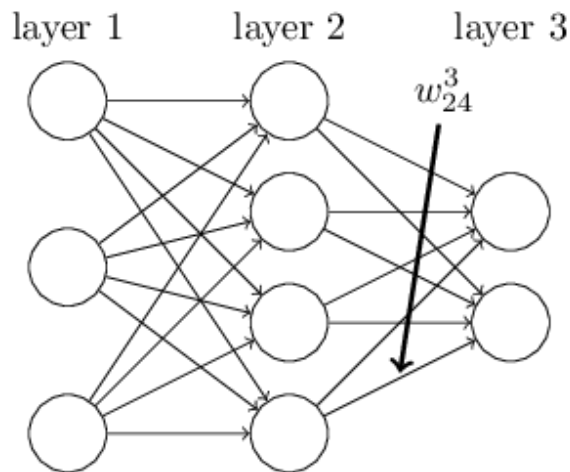


Universal Approximation Theorem

<http://neuralnetworksanddeeplearning.com/chap4.html>

Any complex function can be approximated with a single layer NN with enough neurons

Some ugly notation

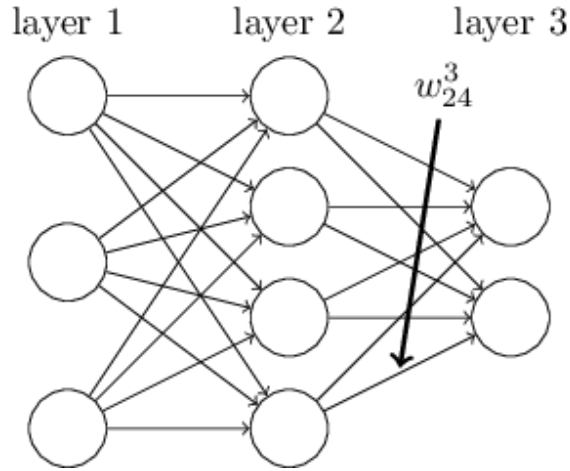


w_{jk}^l is the weight from the k^{th} neuron in the $(l-1)^{\text{th}}$ layer to the j^{th} neuron in the l^{th} layer

$$a_j^l = \sigma \left(\sum_k w_{jk}^l a_k^{l-1} + b_j^l \right),$$

- This will make sense very soon
 - a_j^l = output of j^{th} neuron in l^{th} layer
 - z_j^l = stuff inside the sigmoid
- $$a_j^l = \sigma(z_j^l)$$

Some better notation



w_{jk}^l is the weight from the k^{th} neuron in the $(l-1)^{\text{th}}$ layer to the j^{th} neuron in the l^{th} layer

$$a^l = \sigma(w^l a^{l-1} + b^l).$$

- This notation should make sense now
- w^l = weight matrix for layer l
 - $W_{ij} \rightarrow$ weight from j^{th} to i^{th} neuron
- We can compute an entire batch

Cost Function

$$C = \frac{1}{2} \|y - a^L\|^2 = \frac{1}{2} \sum_j (y_j - a_j^L)^2,$$

- This is the cost for a single training example
- L = last layer
- Must be easily differentiable
- Cost over a batch can be written as average of individual costs (at least in this case)

Gradient Descent

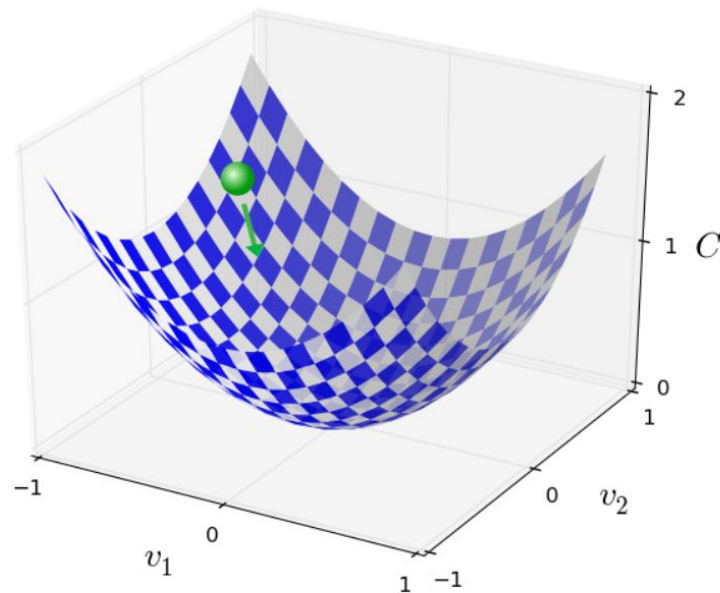
- Just move in the opposite direction of the gradient, duh

$$\Delta C \approx \frac{\partial C}{\partial v_1} \Delta v_1 + \frac{\partial C}{\partial v_2} \Delta v_2.$$

$$\Delta C \approx \nabla C \cdot \Delta v.$$

$$\Delta v = -\eta \nabla C,$$

- Stochastic Gradient Descent: update one batch at once
- Other optimisers: Momentum, RMSProp (reading assignment)



$$a^l = \sigma(w^l a^{l-1} + b^l).$$

Backpropagation

- 1) Calculate the **error** (new term alert) for all layers

$$\delta_j^l \equiv \frac{\partial C}{\partial z_j^l}.$$

- 2) Use the error to get gradients of weights and biases

$$\frac{\partial C}{\partial b_j^l} = \delta_j^l. \quad \rightarrow \quad \frac{\partial C}{\partial b} = \delta,$$

$$\frac{\partial C}{\partial w_{jk}^l} = a_k^{l-1} \delta_j^l.$$

- 3) Apply gradient descent!

Backpropagation - 4 equations

Summary: the equations of backpropagation

$$\delta^L = \nabla_a C \odot \sigma'(z^L) \quad (\text{BP1})$$

$$\delta^l = ((w^{l+1})^T \delta^{l+1}) \odot \sigma'(z^l) \quad (\text{BP2})$$

$$\frac{\partial C}{\partial b_j^l} = \delta_j^l \quad (\text{BP3})$$

$$\frac{\partial C}{\partial w_{jk}^l} = a_k^{l-1} \delta_j^l \quad (\text{BP4})$$

[Proof for BP2](#)

Gradients via recursion

It doesn't need to be so complex

We can break down any expression into simple operations and use chain rule to find the gradient of anything.

Code implementation requires **Object Oriented Programming** → we create a Value class to store gradients

ASSIGNMENT

Implement a class `ValueMatrix`, that instead of storing numbers stores a matrix (as numpy `ndarray`)

Additional operation → matrix multiplication (@)

Try to make the class **inherit** the `ndarray` class